

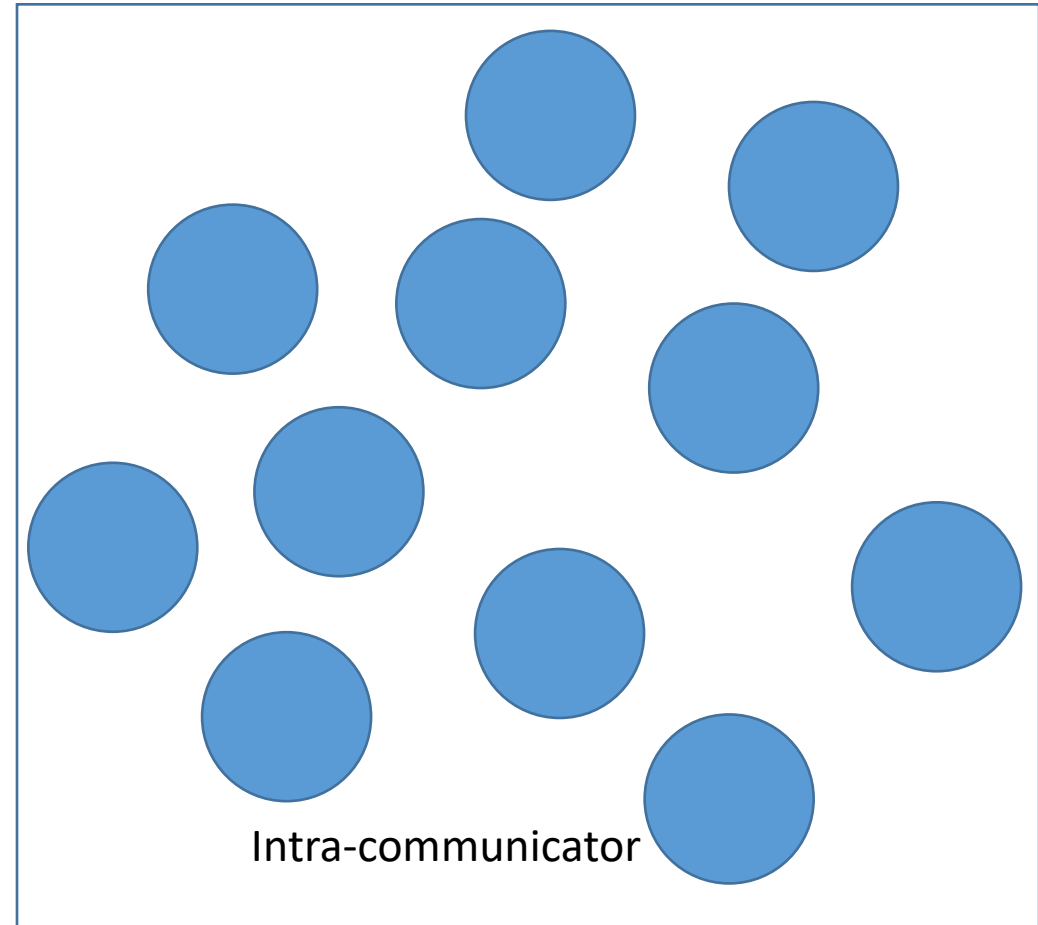
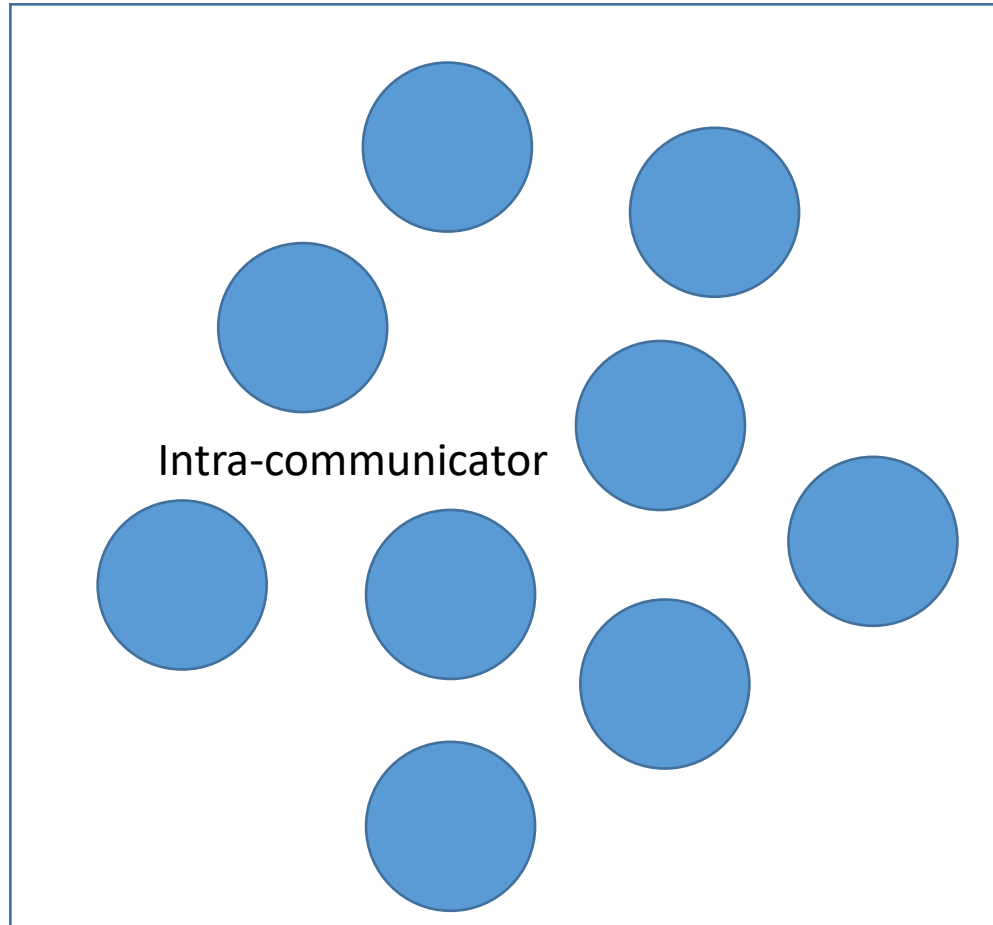
Domain Decomposition

Lecture 11

February 11, 2026

Intra-Communicators

- Logical subset
- Different contexts



MPI_Comm_Split Recap

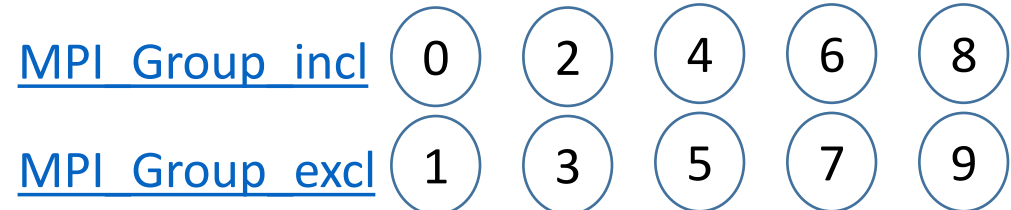
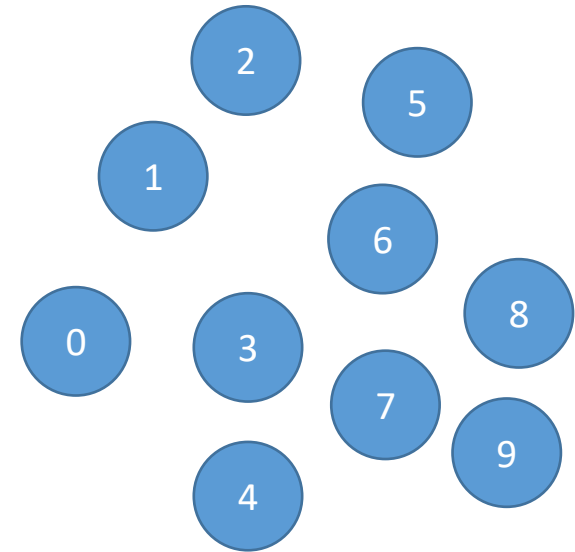
```
MPI_Comm_split (  
    MPI_Comm oldcomm, int color, int key,  
    MPI_Comm *newcomm  
)
```

- Collective call (every process belonging to the oldcomm has to participate)
- Logically divides based on **color**
 - Processes of same color form a group
- Rank assignment based on key
- Output: newcomm

MPI_Group

```
MPI_Group_incl (  
    MPI_Group g_group,  
    int N, // size of the new group  
    const int ranks[],  
    MPI_Group *new_group);
```

```
MPI_Comm_create_group (  
    MPI_Comm comm,  
    MPI_Group new_group,  
    int tag,  
    MPI_Comm *new_comm);
```



MPI_Comm_split Example Code Recap

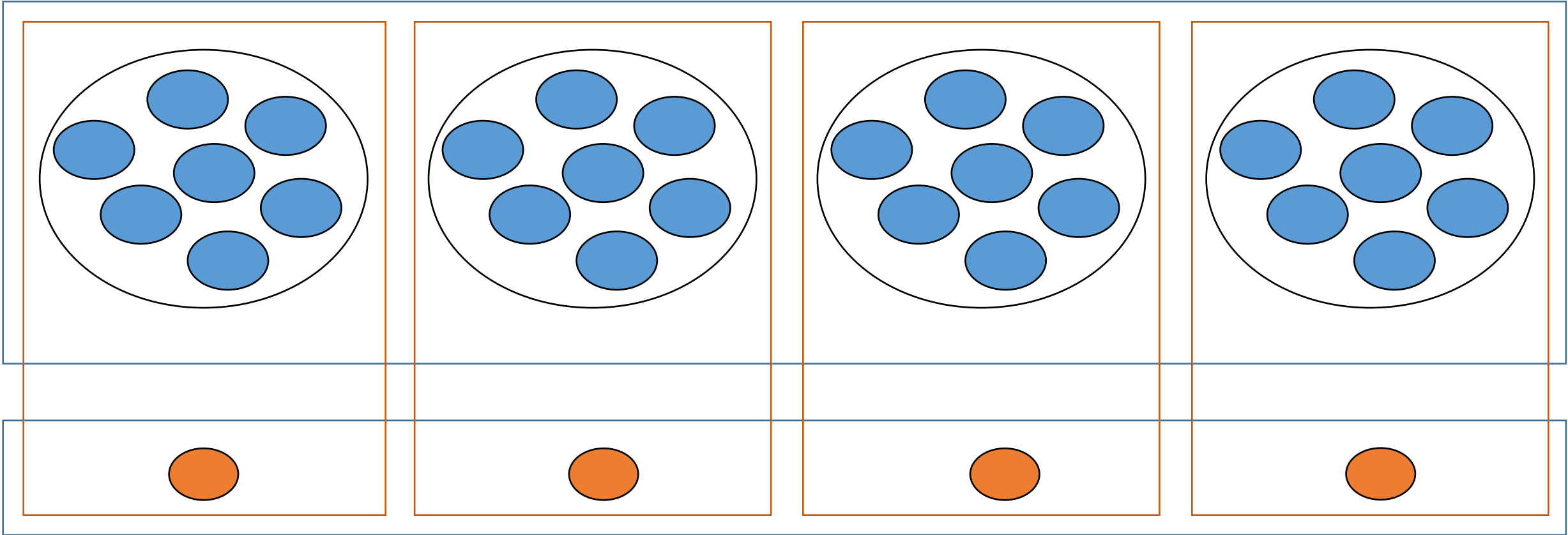
Output
for P=8

```
int newrank, newsize, color = myrank%2;
MPI_Comm newcomm;
MPI_Comm_split (MPI_COMM_WORLD, color, myrank,
&newcomm);
MPI_Comm_rank (newcomm, &newrank);
MPI_Comm_size (newcomm, &newsize);
printf ("%d: %d of %d\n", myrank, newrank,
newsize);
MPI_Comm_free (&newcomm);
```

0: 0 of 4
1: 0 of 4
2: 1 of 4
3: 1 of 4
4: 2 of 4
5: 2 of 4
6: 3 of 4
7: 3 of 4

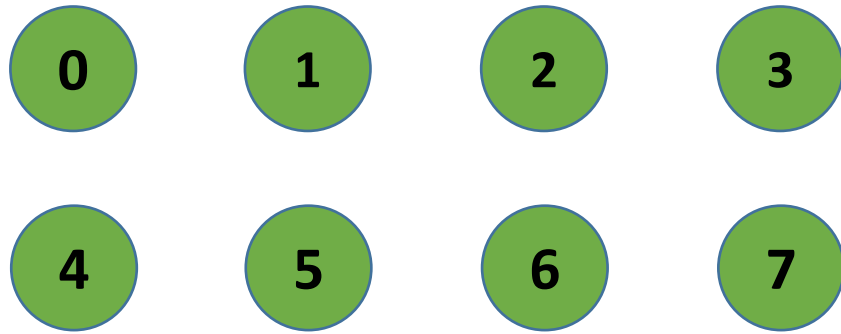
Producer-Consumer Example Recap

An example configuration: 28 simulation processes and 4 analysis processes.

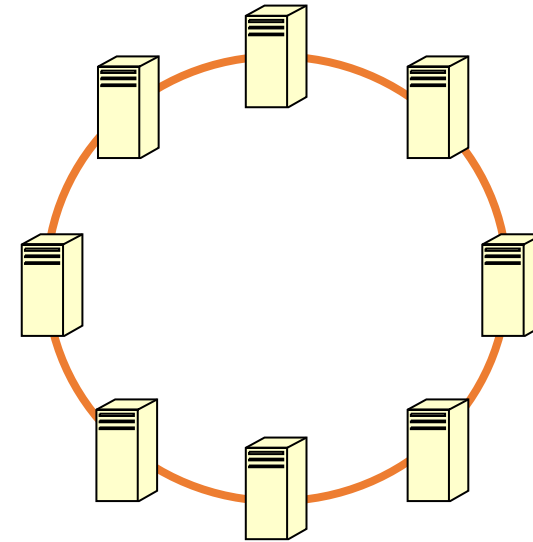


2 sub-communicators suffice: split 1 (blue and orange processes) and split 2 (orange rectangles for each of the four sub-groups)

Process Mapping – Recap



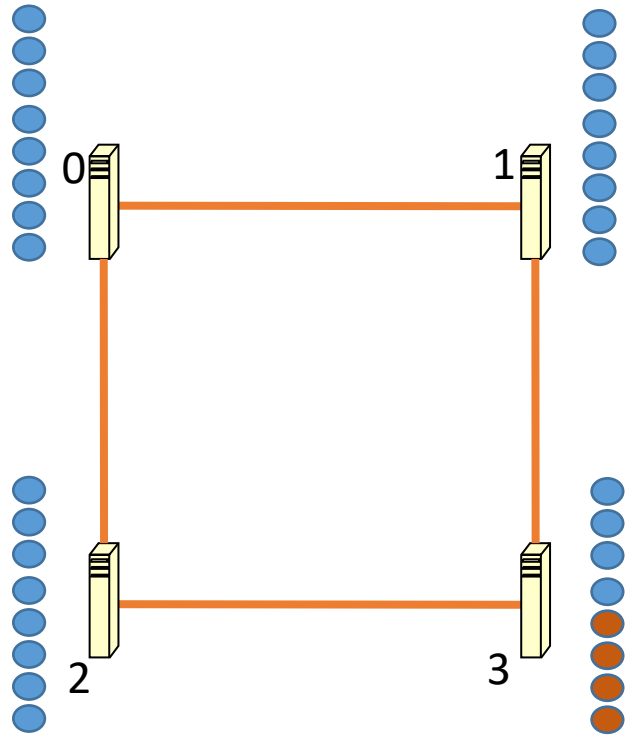
2 X 4 virtual topology



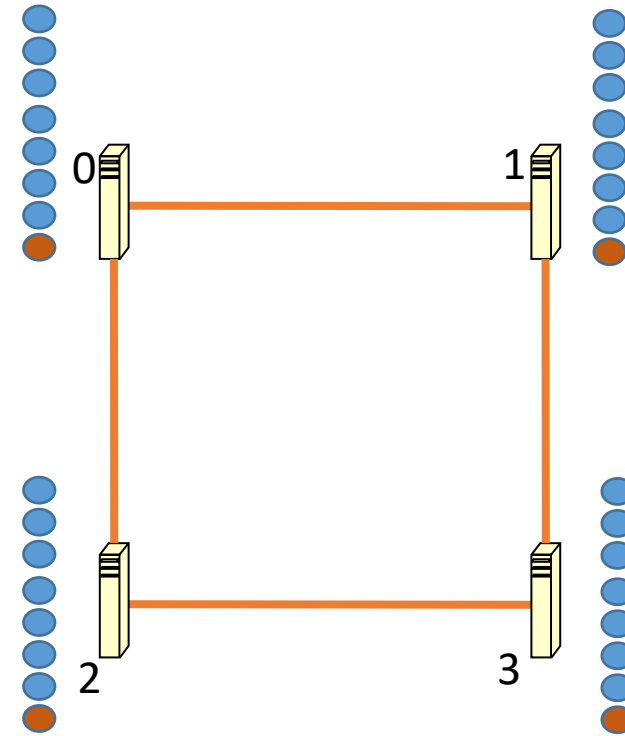
Physical topology

An user may place any rank on any node.

Process Placement for 28:4 Ratio

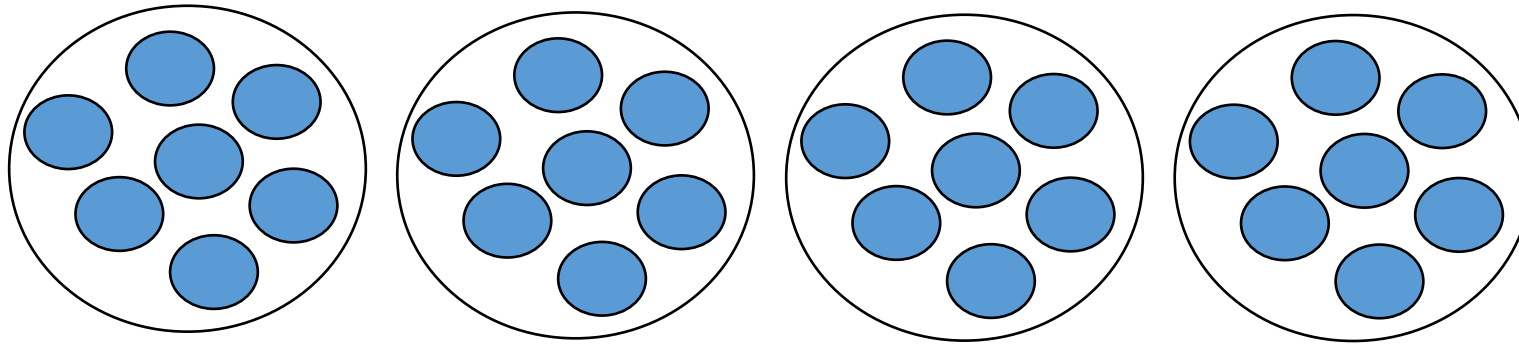


8-core 4-node mini-cluster



8-core 4-node mini-cluster

Process Placement



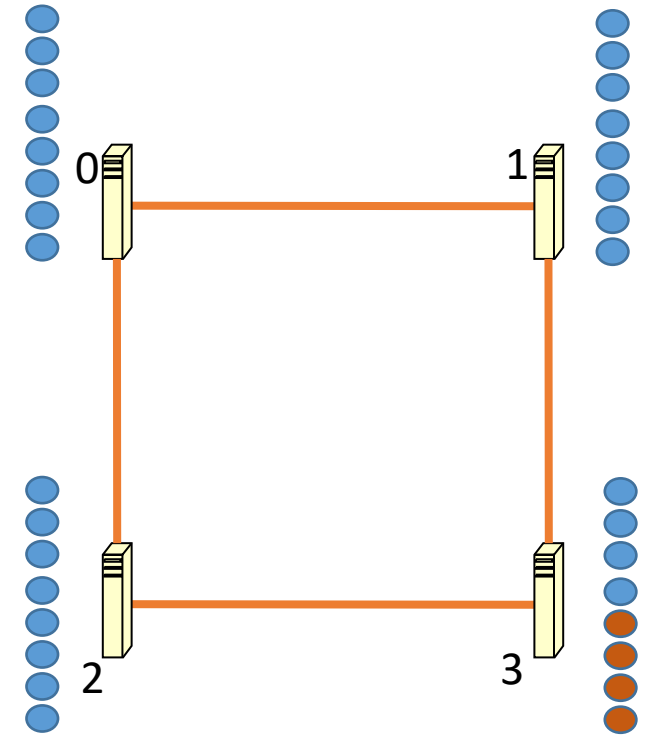
Many-to-one

Many-to-one

Many-to-one

Many-to-one

- How would the communicators be formed?

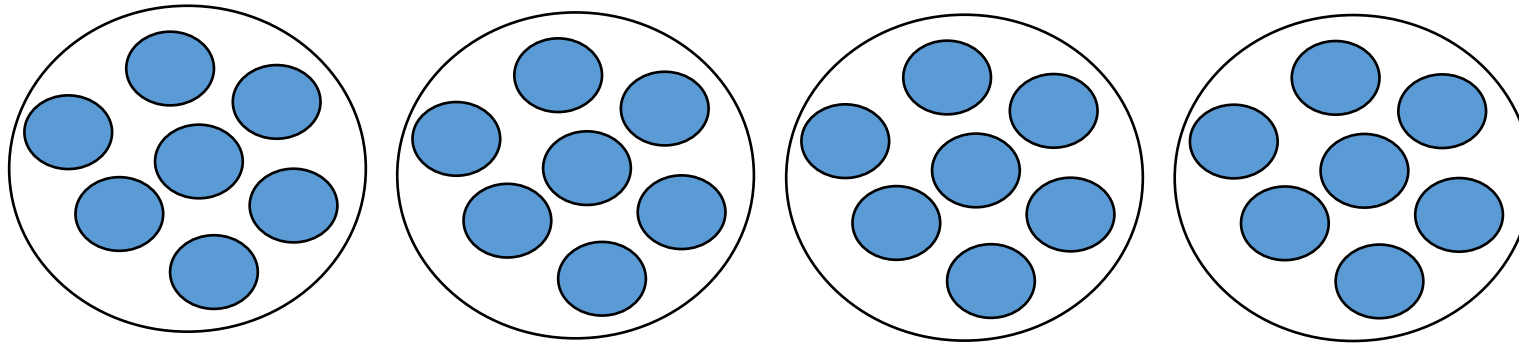


8-core 4-node mini-cluster

Homework

- Write a code for transferring data from a subset (producer processes) to another subset (consumer processes) iteratively. The ratio of producer to consumer processes is an input to the code, along with the data size.

Process Placement



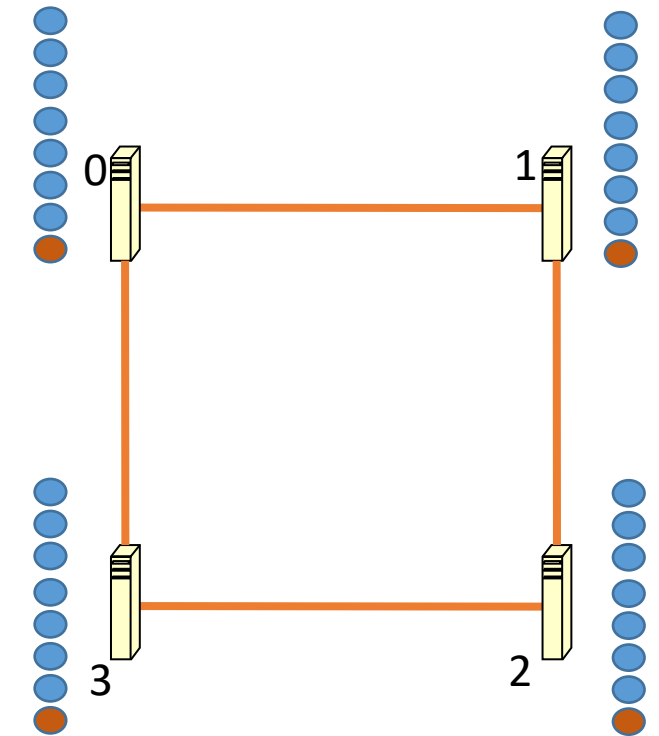
Many-to-one

Many-to-one

Many-to-one

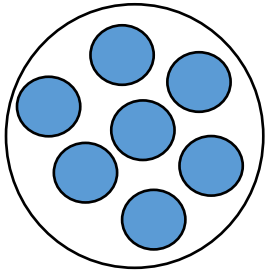
Many-to-one

- How would the communicators be formed?

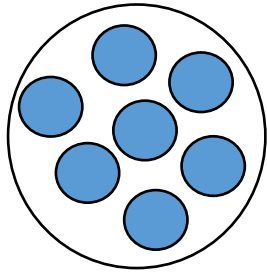


8-core 4-node mini-cluster

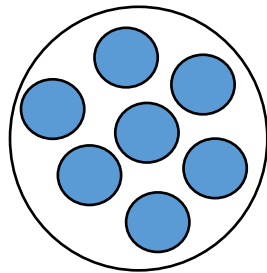
Mapping



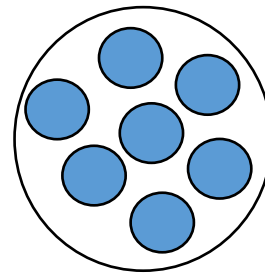
Many-to-one



Many-to-one



Many-to-one



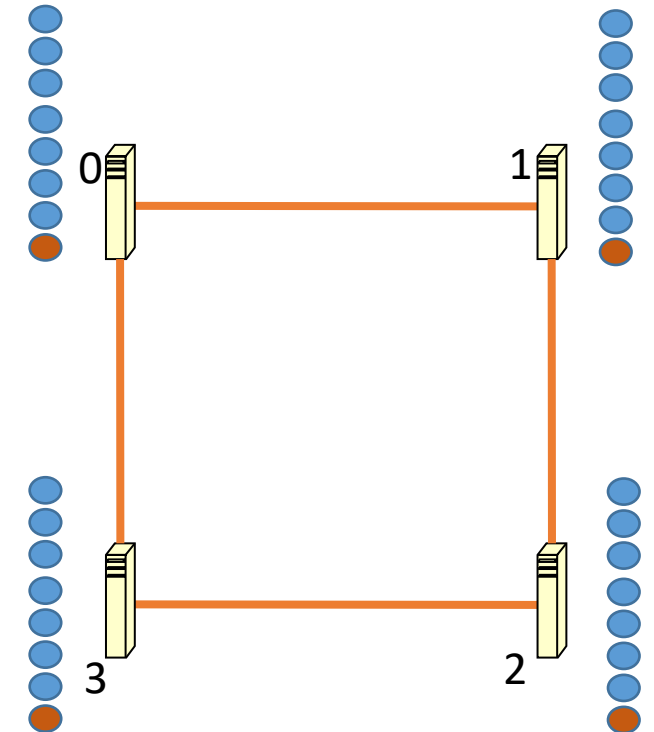
Many-to-one



Node0:7
Node1:7
Node2:7
Node3:7
Node0
Node1
Node2
Node3

An example is shown on the right, ranks 0 – 27 are simulation ranks, 28 – 32 are analysis ranks. One may place ranks 0 – 6 on node 0, ranks 7 – 13 on node 1, ranks 14 – 20 on node 2, ranks 21 – 27 on node 3 using the process mapping file (mapfile) shown on the left. Ranks 28 – 31 are placed on all 4 nodes. The sub-communicators are required to be created accordingly.

- What would be the rank placements?



8-core 4-node mini-cluster

Virtual Topology

0	1	2	3	4	5	6	7	0	4	8	12	16	20	24	28
8	9	10	11	12	13	14	15	1	5	9	13	17	21	25	29
16	17	18	19	20	21	22	23	2	6	10	14	18	22	26	30
24	25	26	27	28	29	30	31	3	7	11	15	19	23	27	31

XY decomposition (2D)

YX decomposition (2D)

4 x 8 2D virtual process topology

Sub-communicator Example

0	1	2	3	4	5	6	7
8	9	10	11	12	13	14	15
16	17	18	19	20	21	22	23
24	25	26	27	28	29	30	31

4 x 8 2D virtual process topology

Question:
Write the code to split row-wise

Input: 2D topology (rows, cols)

Hint: `color = myrank/cols;`

```

// create new group ranks array
int ranks[cols], i, j=-1;
for (i=(myrank/cols)*cols; i<((myrank/cols)+1)*cols; i++)
    ranks[++j] = i;

// get the world group
MPI_Group g_group;
MPI_Comm_group (MPI_COMM_WORLD, &g_group);

// create new groups
MPI_Group new_group;
MPI_Group_incl (g_group, cols, ranks, &new_group);

// create new communicators
MPI_Comm new_comm;
MPI_Comm_create_group (MPI_COMM_WORLD, new_group, myrank/cols, &new_comm); //note the tag

// size of new communicators
int new_size, new_rank;
MPI_Comm_size (new_comm, &new_size);
MPI_Comm_rank (new_comm, &new_rank);
printf ("Old rank %d, new rank %d\n", myrank, new_rank);

```

Sub-communicator Example

0	1	2	3	4	5	6	7
8	9	10	11	12	13	14	15
16	17	18	19	20	21	22	23
24	25	26	27	28	29	30	31

4 x 8 2D virtual process topology

Homework: Split column-wise

Input: 2D topology (rows, cols)

Hint: `color = myrank%cols;`

Sub-communicator Example

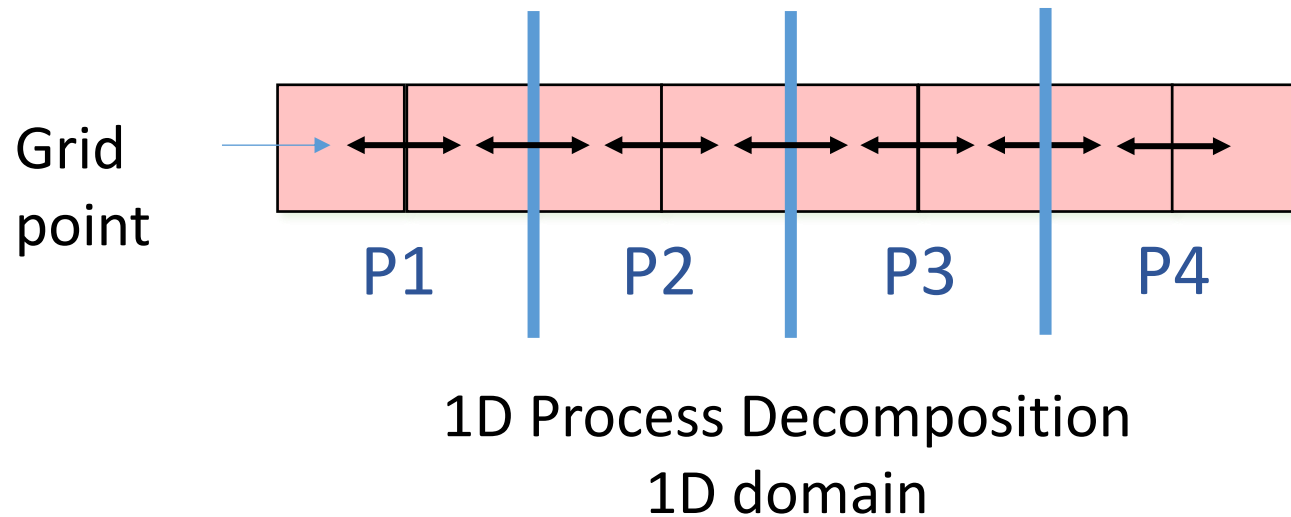
0	1	2	3	4	5	6	7
8	9	10	11	12	13	14	15
16	17	18	19	20	21	22	23
24	25	26	27	28	29	30	31

4 x 8 2D virtual process topology

Homework: Every process computes local array statistics (sum/average). Select 1 process per row to write row-wise cumulative statistics to a file.

Input: 2D topology (rows, cols)

1D Domain Decomposition of a 1D Domain



N grid points

P processes

N/P points per process

#Communications? 2

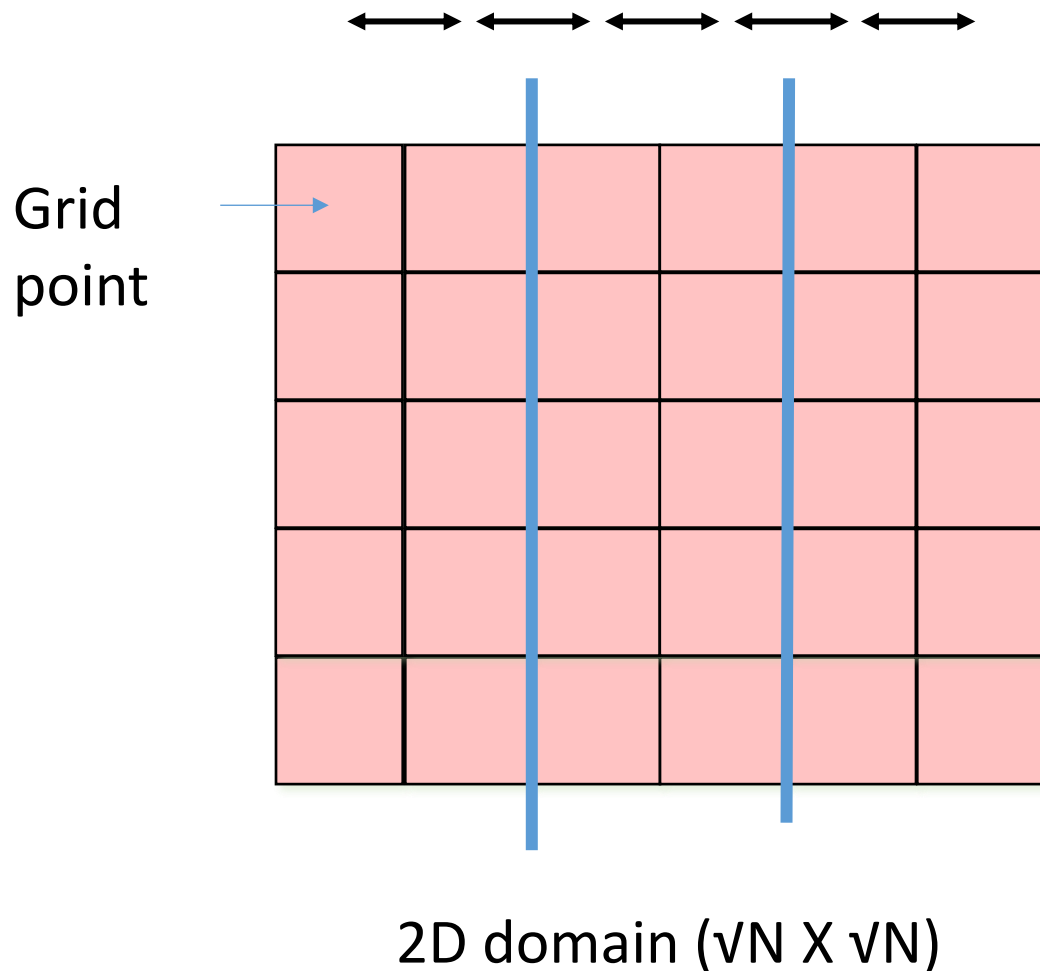
#Computations? N/P

Nearest
neighbor
communications

2 sends()
2 recvs()

Communication to computation ratio = $2P/N$

1D Domain Decomposition of a 2D Domain



N grid points

P processes

N/P points per process

#Communications?

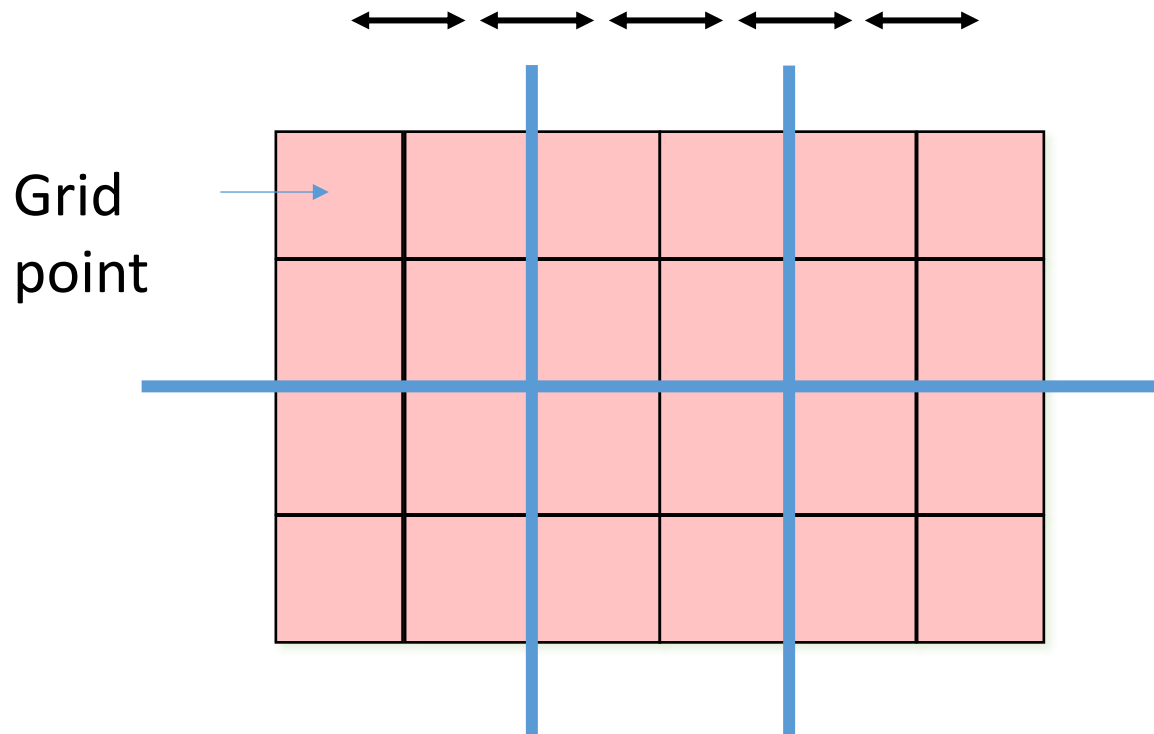
$2\sqrt{N}$ (assuming square grid)

#Computations?

N/P (assuming square grid)

Communication to computation ratio=
 $2P/\sqrt{N}$

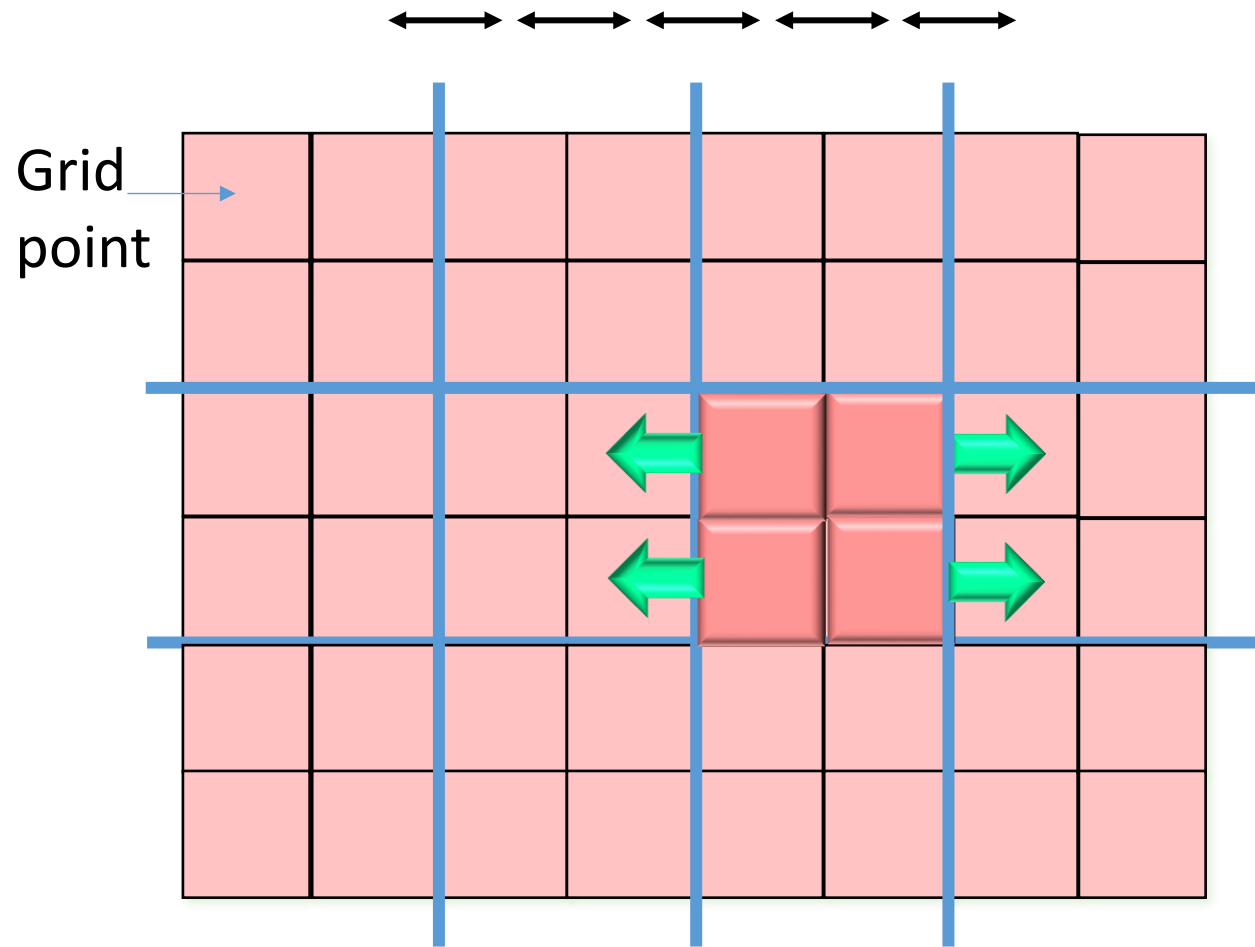
2D Domain decomposition



N grid points ($\sqrt{N} \times \sqrt{N}$ grid)
 P processes ($\sqrt{P} \times \sqrt{P}$ grid)
 N/P points per process

- + Several parallel communications
- + Lower communication volume/process

2D Domain decomposition



2 Sends()
2 Recvs()

#Communications?

$2\sqrt{N}/\sqrt{P}$ (assuming square grid)

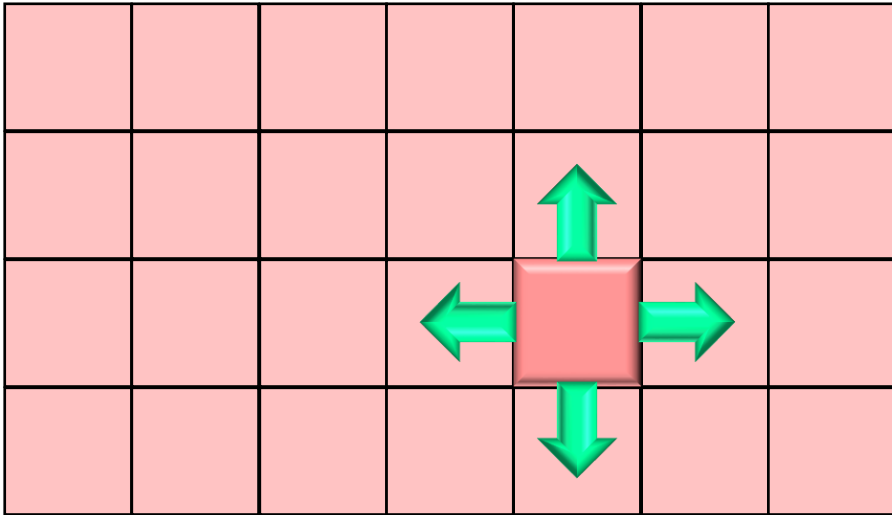
#Computations?

N/P (assuming square grid)

Communication to computation ratio=
 $2\sqrt{P}/\sqrt{N}$

Stencils

$$F^{t+1}(i, j) = [F^t(i-1, j) + F^t(i+1, j) + F^t(i, j) + F^t(i, j-1) + F^t(i, j+1)]/5$$

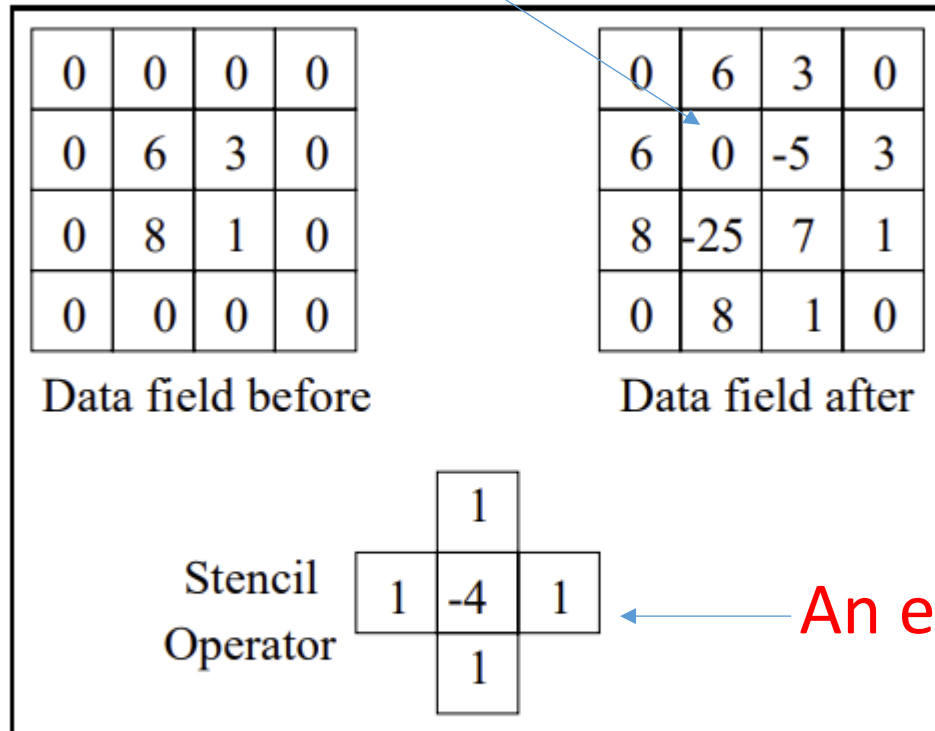


Five-point stencil

- Iterative update of each grid point values
- Values of every grid point at $t+1^{\text{th}}$ iteration depends on the values of its four nearest neighbours at t^{th} iteration

Another Example of 5-point Stencil

Note: this value is supposed to be -13 (there may have been a typo in the paper) [Thanks to Swapnil for pointing]



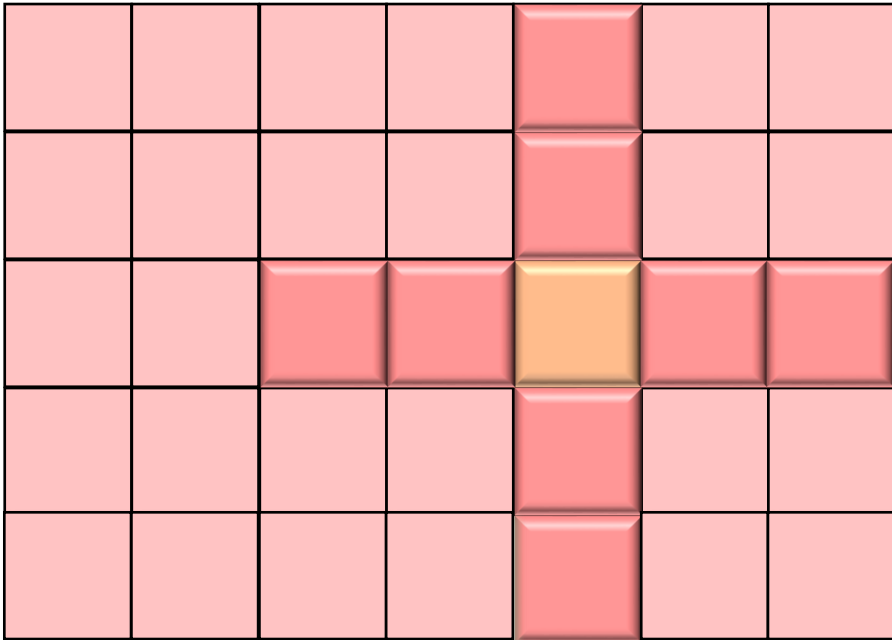
$$\psi_{i,j}^{t+1} = \psi_{i+1,j}^t + \psi_{i-1,j}^t - 4\psi_{i,j}^t + \psi_{i,j+1}^t + \psi_{i,j-1}^t$$

- Iterative update of each grid point values
- Values of every grid point at $t+1^{\text{th}}$ iteration depends on the values at t^{th} iteration

Figure 1: Application of 5-point stencil operator to 2d data field

Source: Stencil Methods on Distributed High Performance Computers, James et al., 1997

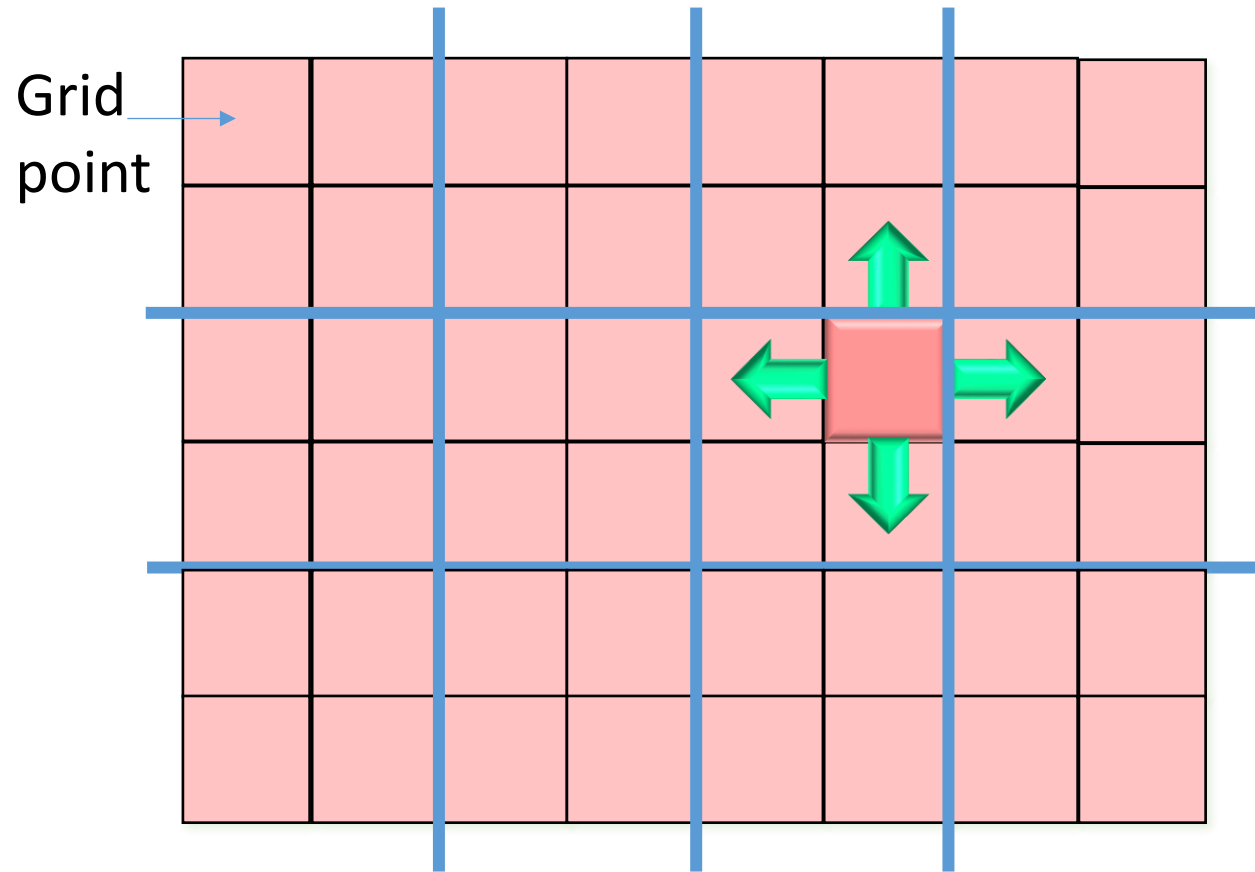
Stencils



9-point stencil

- Iterative update of each grid point values
- Values of every grid point at $t+1^{\text{th}}$ iteration depends on the values of its 9 neighbours at t^{th} iteration

2D Domain Decomposition of a 2D Domain



4 Sends()
4 Recvs()

N grid points ($\sqrt{N} \times \sqrt{N}$ grid)
 P processes ($\sqrt{P} \times \sqrt{P}$ grid)
 N/P points per process

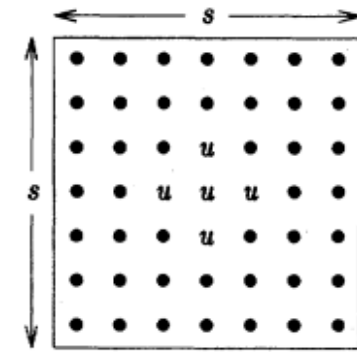
#Communications?

$4\sqrt{N}/\sqrt{P}$ (assuming square grid)

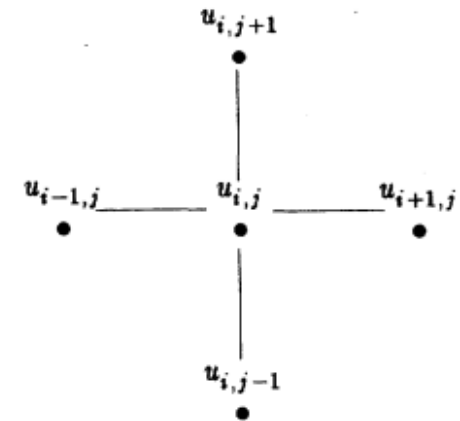
#Computations?

N/P (assuming square grid)

Communication to computation ratio=
 $4\sqrt{P}/\sqrt{N}$



$computation \approx s^2$
 $communication \approx 4s$
 (a)



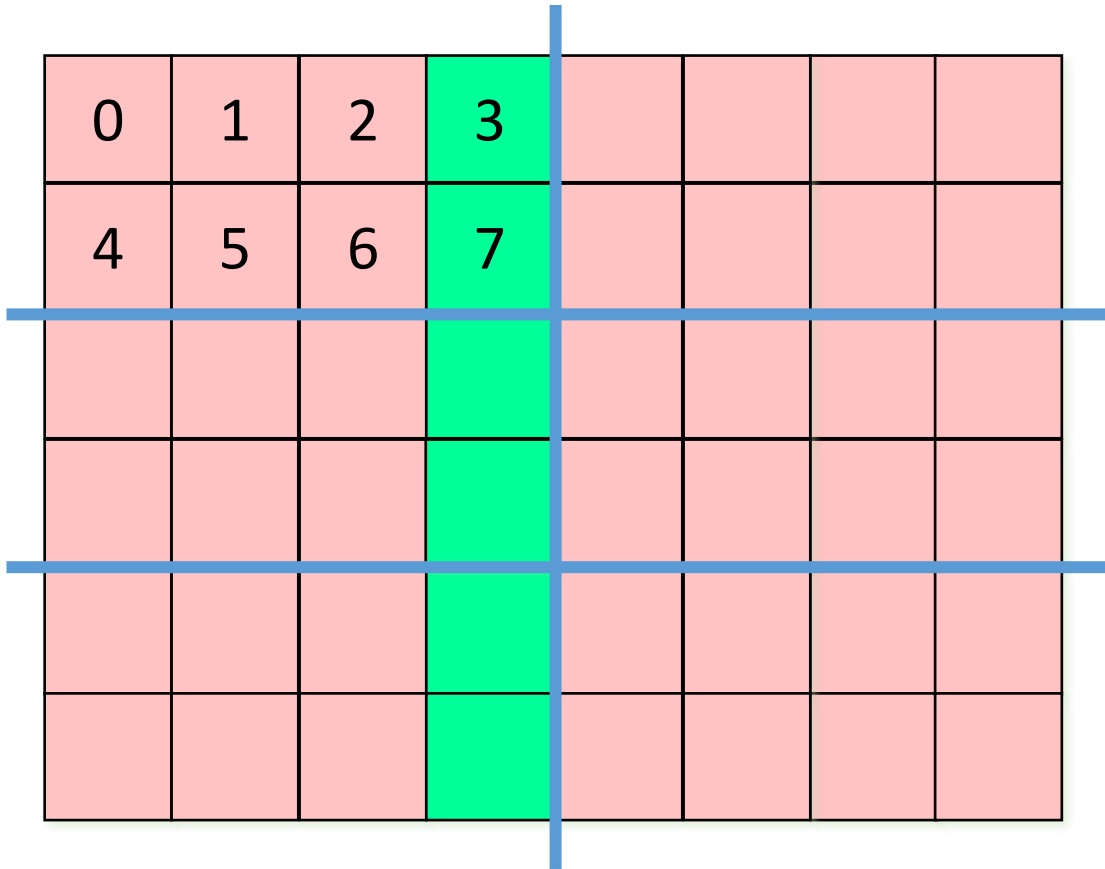
$$u_{i,j} = \frac{1}{4} \left[u_{i-1,j} + u_{i+1,j} + u_{i,j-1} + u_{i,j+1} \right]$$

(b)

Source: Stencils and Problem Partitionings: Their Influence on the Performance of Multiple Processor Systems, IEEE TOC, 1987

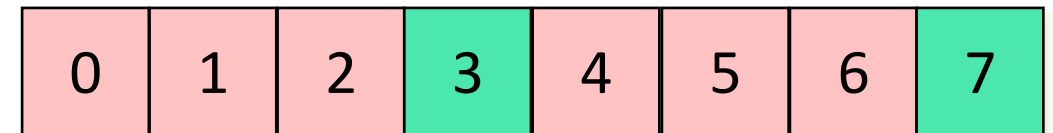
Fig. 1. Square partitions with five-point stencil.

Stencil Implementation - Send / Recv

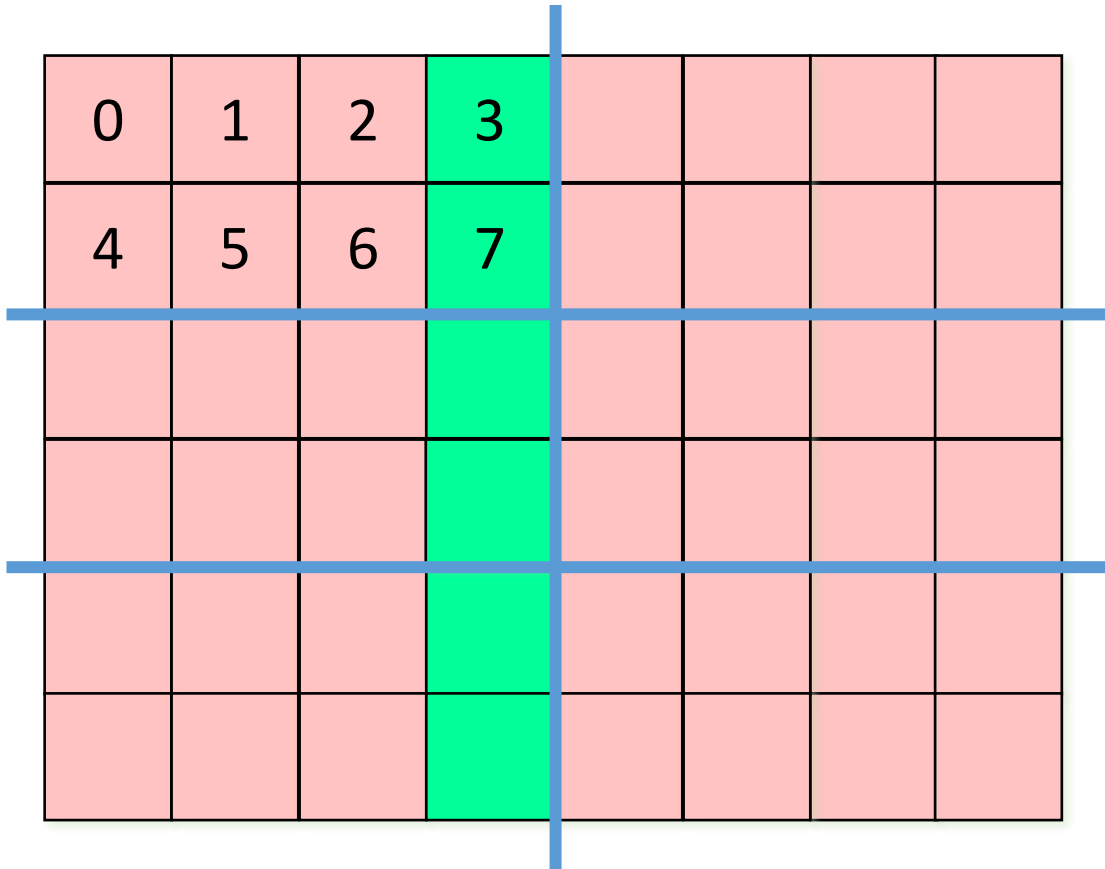


MPI_Send

MPI_Recv

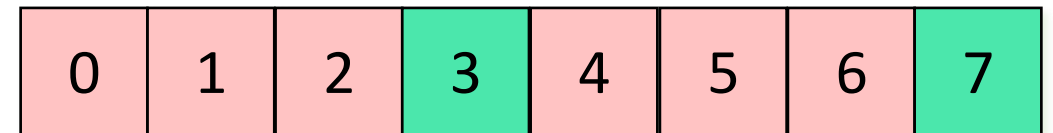


Stencil Implementation - Send / Recv



MPI_Pack (buf)
MPI_Send (buf)

MPI_Recv (buf)
MPI_Unpack (buf)



MPI_Pack

Input Parameters

inbuf

input buffer start (choice)

incount

number of input data items (non-negative integer)

datatype

datatype of each input data item (handle)

outsize

output buffer size, in bytes (non-negative integer)

comm

communicator for packed message (handle)

Input/Output Parameters

position

current position in buffer,
in bytes (integer)

Output Parameters

outbuf

output buffer start (choice)

MPI_Pack

```
MPI_Init(&argc, &argv);

MPI_Comm_rank(MPI_COMM_WORLD, &myrank) ;
MPI_Comm_size(MPI_COMM_WORLD, &size);

// initialize data
for (int i=0; i<M; i++)
    for (int j=0; j<N; j++)
        array2D[i][j] = myrank+i+j;

sTime = MPI_Wtime();
if (myrank == 0) {
    // pack the last element of every row (N ints)
    for (int j=0; j<N; j++) {
        MPI_Pack (&array2D[j][M-1], 1, MPI_INT, buffer, 400, &position, MPI_COMM_WORLD);
        printf ("packed %d %d\n", j, position);
    }
    MPI_Send (buffer, position, MPI_PACKED, 1, 1, MPI_COMM_WORLD);
}
else {
    // receive N ints
    if (myrank == 1)
        MPI_Recv (buffer, count, MPI_INT, 0, 1, MPI_COMM_WORLD, &status);
    // verify
    MPI_Get_count (&status, MPI_INT, &count);
}
eTime = MPI_Wtime();
time = eTime - sTime;

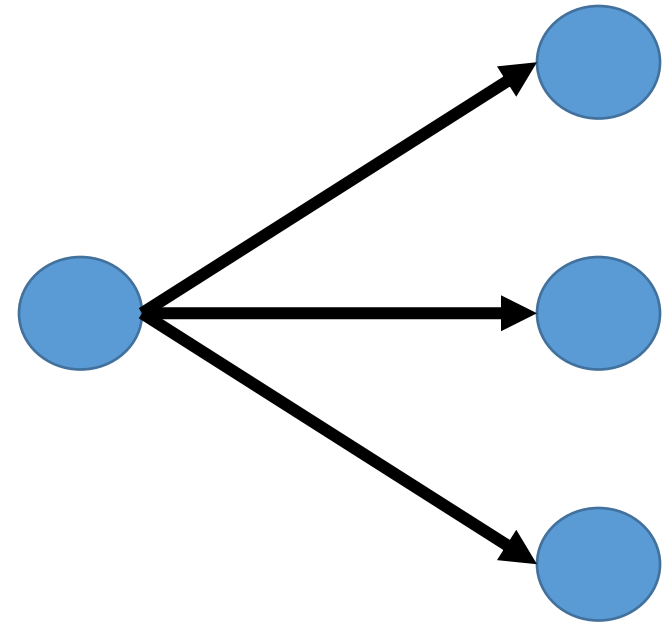
printf ("%lf\n", time);
```

int MPI_Pack (const void
*inbuf, int incount,
MPI_Datatype datatype,
void *outbuf, int outsize,
int *position, MPI_Comm
comm)

```
if (myrank == 0) {  
    // pack num_elem last elements of every row (N * num_elem ints)  
    for (int j=0; j<N; j++) {  
        MPI_Pack (&array2D[j][M-num_elem], num_elem, MPI_INT, buffer, 400, &position,  
MPI_COMM_WORLD);  
        printf ("packed %d %d\n", j, position);  
    }  
    MPI_Send (buffer, position, MPI_PACKED, 1, 1, MPI_COMM_WORLD);  
}  
else {  
    // receive N ints  
    count = N*num_elem;  
    if (myrank == 1)  
        MPI_Recv (buffer,count, MPI_INT, 0, 1, MPI_COMM_WORLD, &status);  
}
```

```
mpirun -np 2 ./pack 4 4  
packed 0 8  
packed 1 16  
packed 2 24  
packed 3 32
```

P2P and Collective



Collective Communication

- Must be called by all processes that are part of the communicator

Types

- Synchronization (MPI_Barrier)
- Global communication (MPI_Bcast, MPI_Gather, ...)
- Global reduction (MPI_Reduce, ..)

Barrier

- `MPI_Barrier (comm)`
- Every rank needs to call this function (for true synchronization)
- Caller returns only after all processes have entered the call

```
printf ("Before barrier");
```

```
MPI_Barrier (MPI_COMM_WORLD);
```

```
printf ("After barrier");
```

Barrier

$n=4$

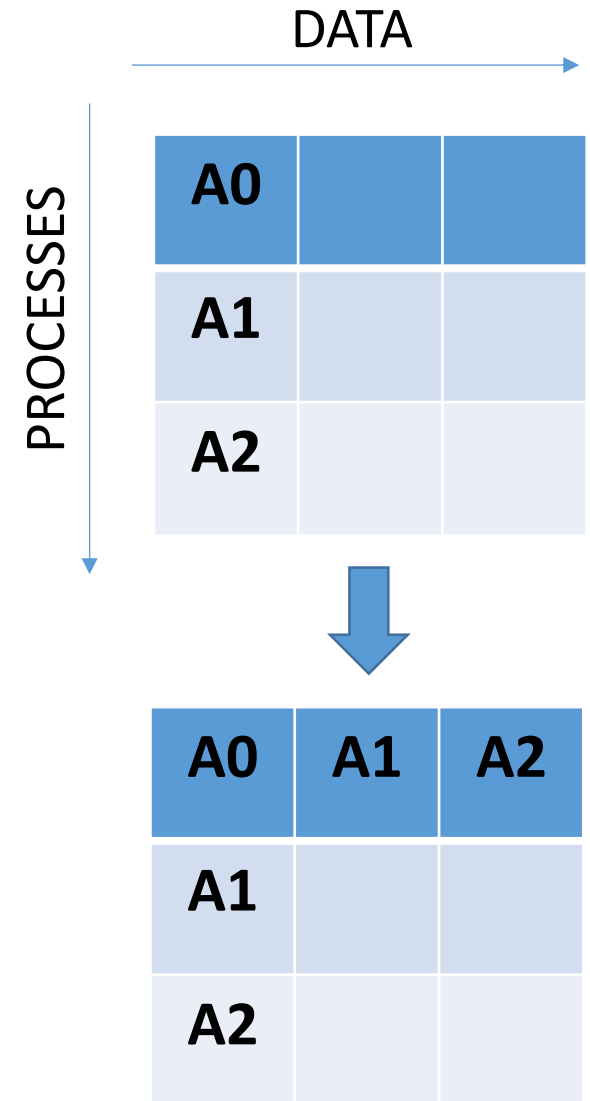
```
if (myrank != 0)
    MPI_Barrier (MPI_COMM_WORLD);
printf("%d\n", rank);
```

Gather

- Gathers values from all processes to a root process
- int `MPI_Gather` (sendbuf, sendcount, sendtype, *recvbuf*, recvcount, recvtype, `root`, comm)
- Arguments `recv*` not relevant on non-root processes
- `recvcount` → size of any (single) receive
- Distinct values (may be same) received from non-root processes at the root process
- Example: Reading multiple files (1 file per process)

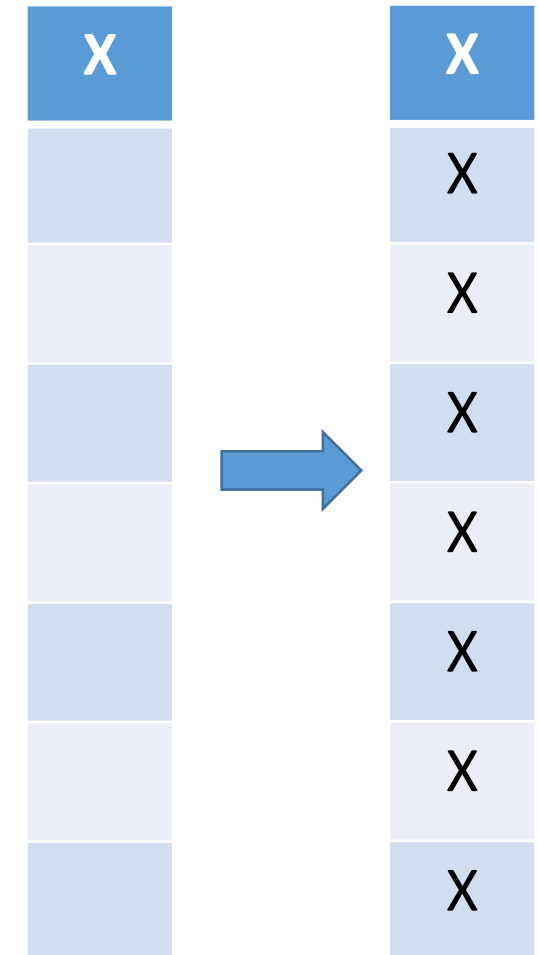
Q: Equivalent point-to-point communications for the same?

- `MPI_Recv` at root
- `MPI_Send` at non-root



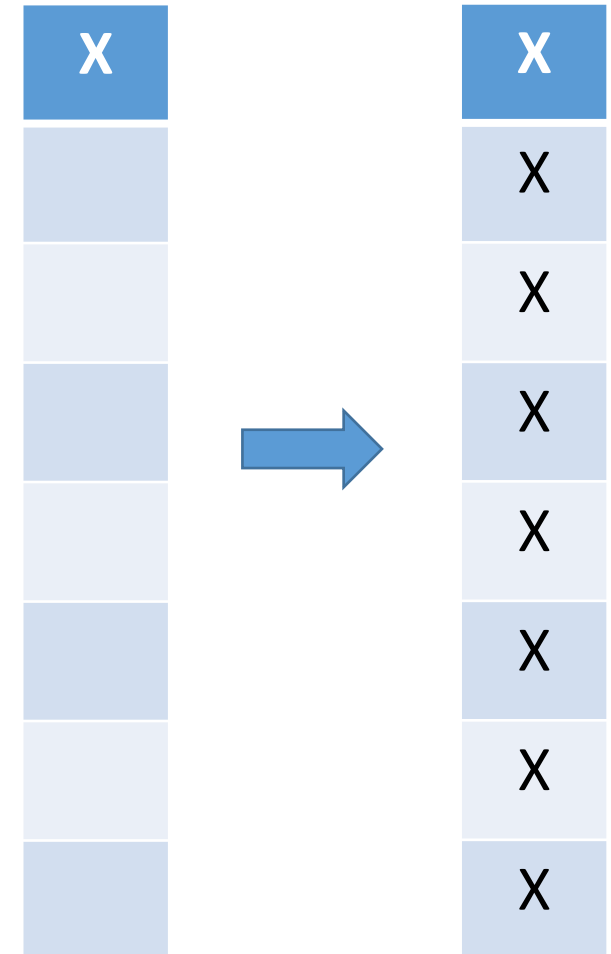
Broadcast

- Root process sends message to all processes
- int `MPI_Bcast` (buffer, count, datatype, root, comm)
- “count” is the number of elements in “buffer” – must match
 - “message sizes do not match across processes in the collective routine: Received 400 but expected 4000”
- “buffer” is input at root, output at non-root
- Any process can be a root process but has to be the same process when `MPI_Bcast` is called



Broadcast

Q1: Point-to-point communication for the same?

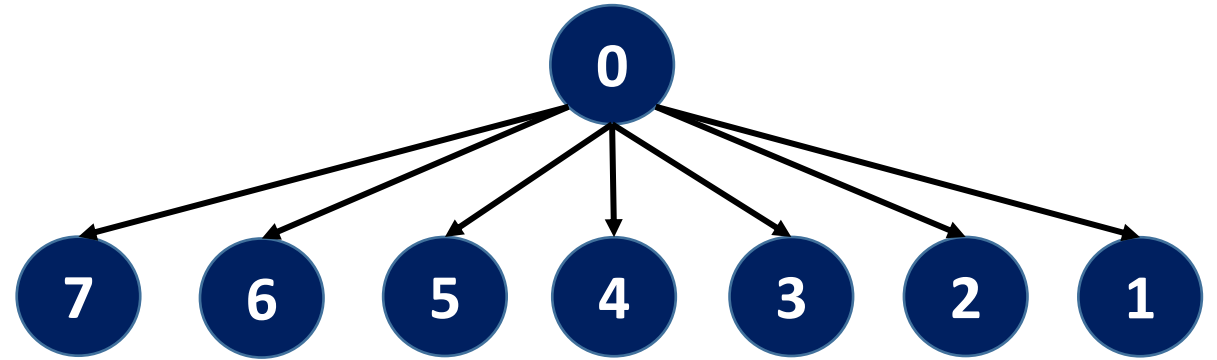


Broadcast – Naïve Algorithm (One-to-many)

- Root process sends to every other process

Cons

- Root is a bottleneck
- Poor scalability
- Idling processes
- Communication links are under-utilized



MPI_Bcast Example

```
// ... initialization tasks
```

```
int color=irand();
```

```
if (myrank != 3) MPI_Bcast (&color, 1, MPI_INT, 0, MPI_COMM_WORLD);
```

```
printf ("%d: %d\n", myrank, color);
```

Output for n=4?

May succeed but not safe

Homework

Write a program to split the total #processes (P) into G groups $\{G_0, G_1, \dots\}$, and the rank $(P/G-1)$ of each group should broadcast its global rank and the global rank of the local rank 0 (as a pair of integers) of the group to all its group members. Assume G is divisible by P .

Scatter

- Scatters values to all processes from a root process
- `int MPI_Scatter (sendbuf, sendcount, sendtype, recvbuf, recvcount, recvtype, root, comm)`
- Arguments `send*` irrelevant in non-root processes
- `sendcount` – Number of elements sent to each process
- Output parameter – `recvbuf`

